generate that URL so that it updates automatically if the routes in your application change.

As you saw in chapter 6, the `LinkGenerator` class is available everywhere in your application, so you can use it inside middleware, minimal API endpoints, or any other services. You can use it from Razor Pages and MVC too, if you want, though the `IUrlHelper` is often more convenient and hides some details of using the `LinkGenerator`.

You've already seen how to generate links to minimal API endpoints with `LinkGenerator` using methods like `GetPathByName()` and `GetUriByName()`. `LinkGenerator` has various analogous methods for generating URLs for Razor Pages and MVC actions, such as `GetPathByPage()`, `GetPathByAction()`, and `GetUriByPage()`, as shown in the following listing.

Listing 14.4 Generating URLs using the `LinkGeneratorClass`

```
public class CurrencyModel : PageModel
{
    private readonly LinkGenerator _link;                          ❶
    public CurrencyModel(LinkGenerator linkGenerator)              ❶
    {                                                              ❶
        _link = linkGenerator;                                     ❶
    }                                                              ❶

    public void OnGet ()
    {
        var url1 = Url.Page("Currency/View", new { id = 5 });      ❷
        var url3 = _link.GetPathByPage(                            ❸
                       HttpContext,                                ❸
                       "/Currency/View",                           ❸
                       values: new { id = 5 });                    ❸
        var url2 = _link.GetPathByPage(                            ❹
                       "/Currency/View",                           ❹
                       values: new { id = 5 });                    ❹
```